

● FINAL PROJECT · FROM LLMS TO AGENTS

PROJECT OFFER ASSISTANT

An in-house offer database with **RAG access**
— searchable · cited · private.

Open Campus

Summer 2026

Ollama

ChromaDB

LlamaIndex

Streamlit

51 real + 10 synthetic offers

100% local

try "Your own terms, found again." |



What we built

A **stateless RAG system** — local inference, ingest to retrieval.

Offer PDF (Text/Image), converted, normalized, PII redacted, facts extracted, saved, multi-level retrieval, cited answers (LLM).

Ollama

vLLM

ChromaDB

SQLite

BM25 + RRF

Streamlit

MODELS

qwen3.8:27b-fp8

qwen3.5:4b-mlx

nomic-embed-text

WHAT IT IS

- two distinct AI workflows
- sovereign data aggregation
- privacy at core

WHAT IT IS NOT

- **not an agent** — no tool loop, no planning
- **not a chatbot** — no conversation memory

WHAT TO EXPECT

- answers from **cited documents**
- links open source in app
- info gaps flagged + **no PII**
- **code-driven behavior**

The Problem

Project-based service providers — across industries. They write **offers** with heterogeneous scope, timelines, milestones, custom contract terms — pages, tailored per client.

digital & IT services

industrial services

agencies

consultancies

PROBLEM 1 — IT TAKES REAL TIME

- a proper offer takes **hours, sometimes a full day**
- clients still expect a fast turnaround
- hard to find how similar cases were scoped, priced, termed

The flip — the value is already there: every past offer is **documented experience** in the archives.

PROBLEM 2 — THE KNOWLEDGE IS TACIT

- delegate offer-writing in a small team → the person needs to know **“how did we usually handle this?”**
- knowledge that normally lives in one person's head only

THE CONSTRAINT client data — **phone numbers, emails, street addresses** — is privacy sensitive.

The Goal

Given that reality, what should a system like this actually do for the people writing these offers?

01 find comparable past offers — **fast**

02 get help **drafting the next offer** from what's already there

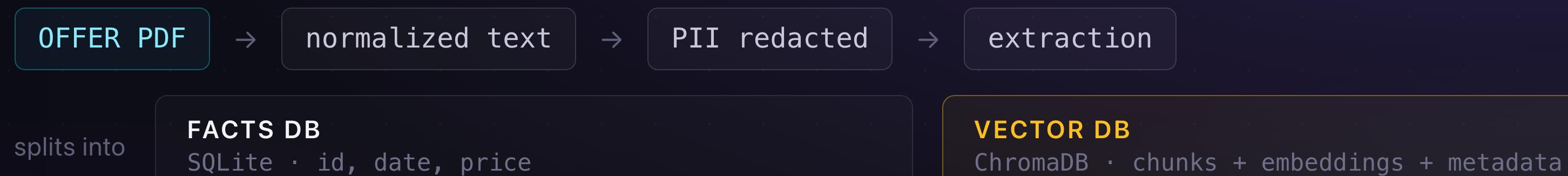
03 surface best practices across the **full breadth** of past offers — not just the closest match

That's the target. On the following pages we take a look at how to get there.

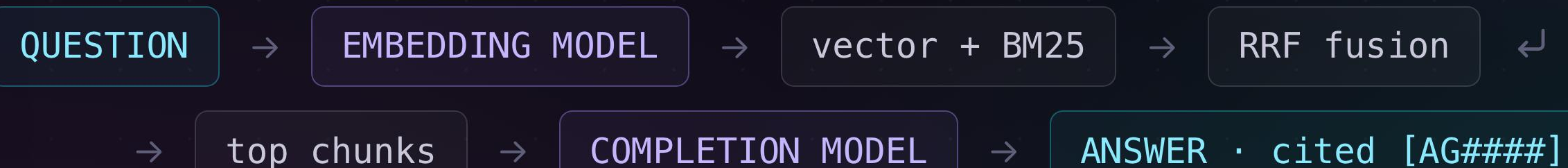
The Data Journey — at a glance

Two halves, one store in the middle — **ingest** runs once per document, **query** runs once per question.

INGESTION — offline, run once



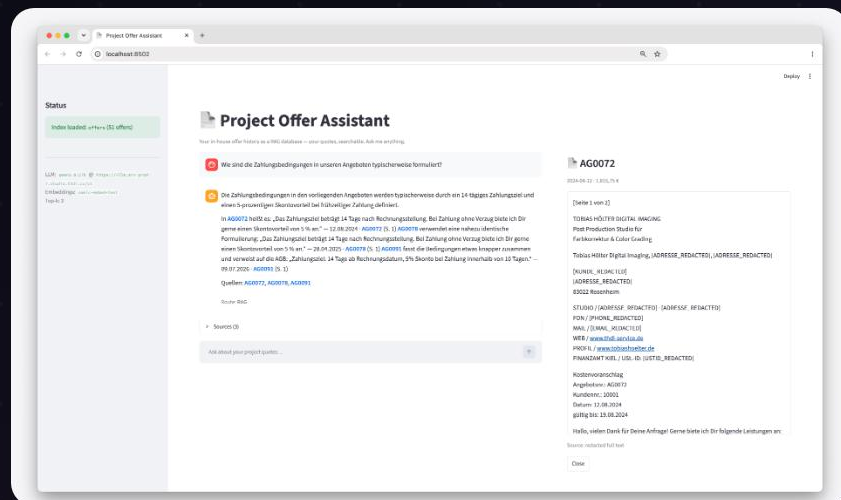
QUERY — at runtime



Let's see it work

One UI, three answer categories: RAG · Clarify · Refusal. How the system picks its path — that's the reveal on slide 14.

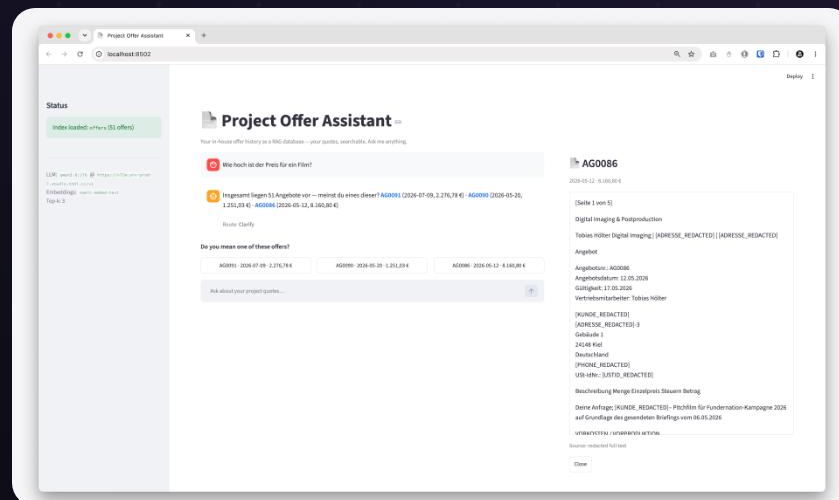
RAG



“Wie sind die Zahlungsbedingungen typischerweise formuliert?”

→ **cited answer** — from 3 offers
AG0072 · AG0078 · AG0091

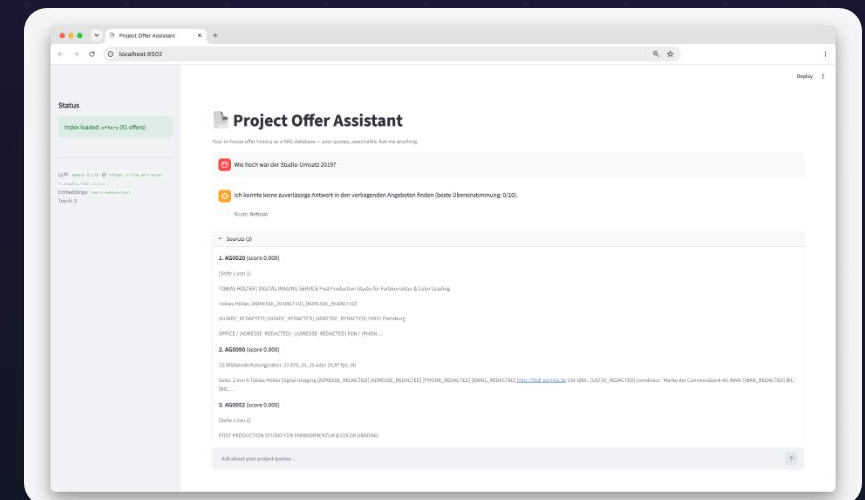
CLARIFY



“Wie hoch ist der Preis für ein Film?”

→ **asks back** — 3 candidate offers
AG0091 · AG0090 · AG0086

REFUSAL



“Wie hoch war der Studio-Umsatz 2019?”

→ **“no documents”** — refusal
best match 0/10

How I approached it

Two pipelines, explored separately — **on test documents**, before touching a single real offer.

INGEST

- test.doc: a **4000-page software manual**
- measured: processing times, stumbling blocks
- minimal retrieval tests
- open question: **PDF vision?** — graphics, context text extraction misses

RETRIEVAL

- test.doc: a **7h38m video transcript**
- the question: how do we retrieve? → **hybrid search**
(**BM25** + vector)
- metadata: how to set it so we get **cited retrievals**
(e.g. where in the video)

THE FIVE FAILURE MODES — THE COURSE'S CHECKLIST

1	bad chunking	1500/300, short offers	solved
2	embedding mismatch	hybrid: keyword + vector	fixed
3	retrieval noise	LLM reranking	fixed
4	context overflow	top-k, big window	n/a
5	hallucination	citations + refusal gate	fixed

Why a vision model — tested, not guessed

Three variants of the same job: turn a scanned offer into clean, complete text.

1b text LLM + layout

layout OK — but in two-column headers, ~20% of dates missed, and the model “fixed” typos on its own.

1c-B raw OCR, no instructions

entire two-column header missing — 59 words dropped, incl. IBAN/BIC. Incomplete.

1c vision

SELECTED

qwen3.8:27b on local vLLM — reads the page as a human would: one model, one architecture for every human-readable doc.

local + private — never leaves the box

one model, one prompt — no per-format branching

raw text = foundation — a trusted translation

Result: 51/51 offers normalized · 0 read errors

The trust layer

Open-source LLMs “**fix**” **typos**, normalize punctuation, re-interpret expressions. One test: “USt-IdNr.:" became “USt-IdNr” — **one dot, silently gone**.

THE ARCHITECTURE

- 01 vision output + `pdfplumber` reference
 - ↓ two sources
- 02 **word-multiset diff** — `pure code`: dropped? added? decimals?
 - ↓ violations
- 03 **repair loop** (max 2) — model gets the `exact token list`, fixes only those
 - ↓ still dirty
- 04 **strict abort** — `no write`, the run stops

Result: 51/51 clean · 0 read errors

WHY IT HOLDS

- the model **transcribes** — code **verifies**. Never the other way.
- the reference is pure code. The diff is pure code.
- a failed offer is **not written** — the whole run aborts.

Principle: the model never grades its own homework.

Clean before it enters — the LLM never masks itself

THE SANITIZER — TWO LAYERS

- 1 **regex** — phone · email · IBAN · BIC · tax IDs
- 2 **LLM block detection** — reports sender + customer verbatim only
- || **stop point** — human review of the detections
- 3 **combined mask** — containment check: only literally-present strings
- 4 **re-scan hard gate** — 0 hits or abort

THEN IT ENTERS THE STORE

- clean text → **extract**: datum, preis — explicit values only
- the LLM **reports** strings — **a script masks**

CHROMA **60 chunks** — anbot_id, abschnitt

SQLITE **51 rows** — anbot_id, datum, preis

World-facts rule: metadata = IDs, dates, prices. Never free text, no open vocabulary.

Result: 172 strings masked · 0 leaks · 51 docs → 60 chunks

The pipeline — built for short, technical documents

An offer is short & technical — little context to lean on, full of codes and terms. **Exact words decide.**

FOUR LAYERS

- 1 **RETRIEVE** — vector (meaning) + BM25 (keywords) → **RRF fusion** → top-10 (+ HyDE, experimental)
↓
- 2 **RERANK** — LLM scores each candidate 0–10 → top-3
↓
- 3 **GATE** — best score < 5 → “not in the documents”
↓
- 4 **ANSWER** — grounded in the 3 chunks, inline **[AG####]**

WHY LAYER 1 IS HYBRID

It's the S7 fix (embedding mismatch) — proven on a canary: a **7h38m foreign-domain transcript**, where keywords and meaning pull in different directions.

CANARY · BEST-CANDIDATE RERANK SCORE

vector-only 3/10 ✗ → + **BM25/RRF** 8/10 ✓

An offer = Beleg + paper trail. Today we index the **Beleg only** — cross-document flips the picture. (Outlook — S17.)

Tested at every stage

Built is not proven — so we tested at every stage. Each test answers a **different question**: honesty, retrieval quality, strategy.

Honesty 51 REAL PDFS	8/8	Does it cite & refuse honestly? Yes — every answer cited, the refusal correct. Basic pipeline — honesty, not retrieval quality.
Retrieval quality HYBRID · SOFTWARE MANUAL	4/9/5	Does it hold up outside its own domain? 4 correct · 9 partial · 5 missed — of the 5, three were honest refusals, two were wrong answers.
Strategy 8 ROUTES	3/7 → 7/7	Does every route work? Plain RAG got 3/7 right — breadth questions came back incomplete (only 3 chunks). Routing fixed all seven answers.

The **3/7** is the turning point — on the 51 real offers, plain RAG silently under-delivered on the breadth questions. So we built the routes. **Next few slides.**

The Turning Point — A Ceiling by Construction

A very good retrieval system — good chunking, fusion, reranking, citations. And then the tests asked it questions it was **never built for**:

- ❏ “How many of my offers used a daily rate?” **top-k can't count**
- ❏ “Compare the pricing across all of my 2025 offers.” **top-k can't compare**
- ❏ “What's my average discount across projects?” **top-k is capped at k**

FINDING In most of my real questions, RAG-as-usually-practiced is the wrong tool.

Routed retrieval — one system, eight strategies

A **deterministic router** reads the question first and picks the strategy — it decides **how** to answer before it answers.

ID / PRICE names AG####
deterministic lookup — or that offer only

DRAFT “new request”
broad retrieval → LLM map → LLM reduce assembles

STATISTICS “how many / average”
full scan → JSON per offer → **code reduce** ← the S13 fix

COMPARISON “compare across”
topic retrieval → LLM map → LLM reduce

YEAR-LIST year + “offers”
deterministic metadata scan

CLARIFY ambiguous
ask back instead of guessing

BASELINE RAG aggregation
standard pipeline + honest limitation note

DEFAULT RAG everything else
hybrid + rerank + gate

Map-reduce routes — one shape, three engines

All three breadth routes share ONE shape: **MAP** — one LLM call per offer → a structured fact line. **REDUCE** — fold the lines into an answer.

ROUTE	MAP (LLM / OFFER)	REDUCE
STATISTICS “how many / average”	JSON facts per offer — paydays, skonto %, ...	☑ code , no LLM : count · mean · span · outliers
COMPARISON “compare across”	one fact line per offer	one LLM call over the lines → comparison
DRAFT “draft a revised offer”	JSON building blocks — positions, sentences, terms	one LLM call assembles the draft — generation, not lookup

DESIGN POINT the reduce step is where small models fail — they guess instead of compute. So the statistics reduce is **code**.

The technical status — at time of submission

No agent framework — a **stateless, deterministic router**. A plain function decides once, then answers. No loop, no memory, no LangGraph (yet).

LAYER	STATUS
SERVICE EDGE	<code>Streamlit</code> only — no separate API layer
CONTRACTS	plain env config — no request schema
"AGENT" CORE	router fn — stateless, no loop, no memory
DURABLE DATA	Chroma + SQLite, joined on offer ID
FAST STATE	in-process cache only — no Redis/TTL
MODEL ACCESS	2-way client switch + per-call retries
OBSERVABILITY	not yet — Langfuse planned, not wired
QUALITY	97 tests in 9 files + Makefile targets

WHY NO LANGGRAPH nothing here **loops or re-plans** — it classifies once. Simpler to test, easier to trust. A loop stays a clean addition, not a rewrite, if a route ever needs one.

Why this was worth it — the context gets bigger

The offer PDF is the surface. The substance is the **process around it**.

TODAY IN THE INDEX

- 51 offers — the documents themselves

TOMORROW

- every email of the offer process
- briefs, change requests, handover notes

10–100× more text per offer

WHAT THAT UNLOCKS

“Why did this position's price change in round 2?”

“What did the client actually ask for, beyond the PDF?”

- analysis the **PDF alone could never answer**

THE HONEST STATUS

- not there yet — **one linked database**, not two stores



The ingest discipline, the citations, the reranking — that was not over-engineering. That was the foundation for this.

In hindsight, I'd scope it smaller

I didn't know what I wanted yet —

and the wanting kept revealing itself, layer by layer:

“a RAG for my offers”

↳ “...that must not leak **PII**”

↳ “...that can be trusted with a **citation**”

↳ “...that knows when it **doesn't know**”

↳ “...that can also **count & compare**”

↳ “...that eventually sees the **whole process**”

I bit off more than I could chew. (The chew is ongoing.)

My own inference engine — AI power on-prem

The ingest pipeline is demanding — and I wanted to run it on **real customer data**, which rules out proprietary APIs. So my post-production workstation got a **night job**.

BUILD

- Ubuntu 24.04 on the 2× RTX 5090 box — 64 GB VRAM
- vLLM — venv + systemd
- Qwen3.8-27B-FP8, TP = 2
- 256k context window

MEASURE FIRST

- Grafana on the vLLM metrics — **before** touching a knob
- decode rate (tok/s) · cache usage
- it told me where the time went

TUNE THE ENGINE

- speculative decoding (MTP) — draft 3, verify
- chunked prefill — smoother batching
- FP8 KV-cache — halves KV memory

RESULT



Where everything runs

GPU BOX — CUSTOM API ENDPOINT

- serves the model (Qwen 3.8 27B) — pure inference engine
- 110–140 tok/s
- Prometheus & Grafana (Docker) — monitoring

LAPTOP — DEBUG & CLIENT CONSOLE

- Ollama — `nomic-embed-text`
- LlamaIndex orchestration · ChromaDB (on disk)
- Langfuse (self-hosted)

glue: **OpenAI-compatible API** · network: **LAN only, no ports out**

01-submission **fictitious data** ,reproducible 02-demonstration **real, redacted** — local only

“Take a look — the repo behind this presentation is on **GitHub**.”

 <https://github.com/SchnitzelKingKong/project-offer-assistant> |



● FINAL PROJECT

Thank you.

Questions?
I'd love them.